

Lecture 21: The Road to Attention

The fix that worked, and the wall it hit

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The disease, recalled

L20 ended on a number: influence fades as V^k , and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

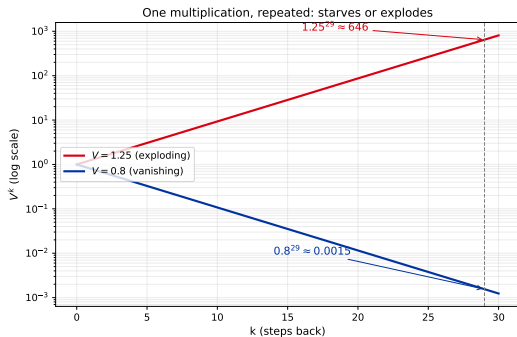
Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

The disease, recalled

L20 ended on a number: influence fades as V^k , and $0.8^{29} \approx 0.0015$. Multiplying the same matrix into itself, 29 times, starved the gradient before it ever reached the first token.

Predict: what arithmetic operation would *not* shrink a signal no matter how many times you repeated it?

Addition. Repeated multiplication decays (or explodes); repeated addition just accumulates. Hold that thought - it is the first half of this lecture, and you have already seen it once (ResNet's skip connection, L17).



Multiplication: the disease. **Addition:** today's medicine.

Outline

LSTM: the additive fix

Tokens: how text becomes numbers

Language modeling

Seq2Seq and the wall

Epilogue: recurrence in 2026

A memory highway

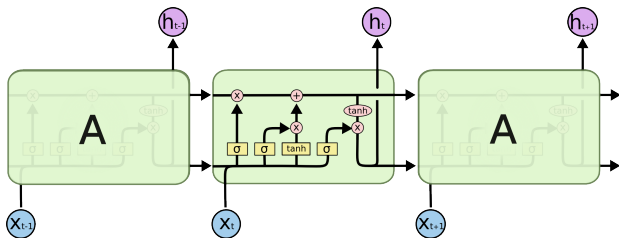
Hochreiter & Schmidhuber's 1997 answer to the disease Hochreiter's own 1991 thesis diagnosed.

The cell state

The **Long Short-Term Memory** (**LSTM**; Hochreiter & Schmidhuber, 1997) adds a second track running alongside the hidden state z : the **cell state c**. At every step it is only lightly modified - a little is removed, a little is added.

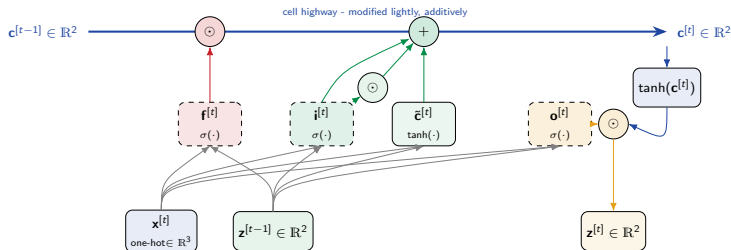
Sorting-line metaphor (ch1-6's world): the belt itself carries context past every station; each worker only adds or removes a little as the box goes by. Nobody rebuilds the box from scratch at every station.

Every horizontal line in the diagram is a full vector; circles are element-wise operations; boxes are learned layers.



Anatomy of one LSTM step

Same toy setup as L20's anatomy frame: a 3-word vocabulary, now a **2-d cell state** alongside the 2-d hidden state. Three gates and one candidate, each a function of $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$; each gate is a sigmoid - a vector in $(0, 1)^2$ used as an **element-wise mask** ($\odot$).

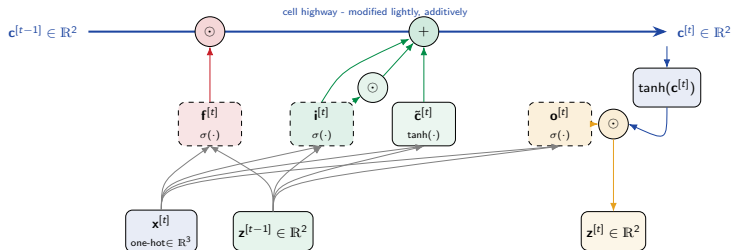


FORGET: drop the old subject on a new one (LMU's gender-pronoun case) INPUT: write down a strong opinion word OUTPUT: what the readout needs right now

Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

Anatomy of one LSTM step

Same toy setup as L20's anatomy frame: a 3-word vocabulary, now a **2-d cell state** alongside the 2-d hidden state. Three gates and one candidate, each a function of $\mathbf{x}^{[t]}$ and $\mathbf{z}^{[t-1]}$; each gate is a sigmoid - a vector in $(0, 1)^2$ used as an **element-wise mask** ($\odot$).



FORGET: drop the old subject on a new one (LMU's gender-pronoun case) INPUT: write down a strong opinion word OUTPUT: what the readout needs right now

Predict: forget gate $[0.9, 0.1]$ meets cell state $[2.0, -1.0]$ - what survives?

$[0.9, 0.1] \odot [2.0, -1.0] = [1.8, -0.1]$: memory 1 survives almost untouched, memory 2 is nearly wiped. A gate is [41_1]'s sigmoid neuron, rebranded as a soft switch.

For the record: the four equations

Reference, not exam material - shown once.

Forget gate: $\mathbf{f}^{[t]} = \sigma(\mathbf{W}_f^\top \mathbf{x}^{[t]} + \mathbf{V}_f^\top \mathbf{z}^{[t-1]} + \mathbf{b}_f)$

Input gate: $\mathbf{i}^{[t]} = \sigma(\mathbf{W}_i^\top \mathbf{x}^{[t]} + \mathbf{V}_i^\top \mathbf{z}^{[t-1]} + \mathbf{b}_i)$
 $\tilde{\mathbf{c}}^{[t]} = \tanh(\mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{b})$

Cell update: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]}$

Output gate: $\mathbf{o}^{[t]} = \sigma(\mathbf{W}_o^\top \mathbf{x}^{[t]} + \mathbf{V}_o^\top \mathbf{z}^{[t-1]} + \mathbf{b}_o)$ $\mathbf{z}^{[t]} = \mathbf{o}^{[t]} \odot \tanh(\mathbf{c}^{[t]})$

The punchline: $\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \dots$ is **additive** - gradients flow through the cell state with minimal decay. ResNet solved *depth* with addition in 2015; LSTM solved *time* with addition in 1997 - same medicine, 18 years earlier.

Why addition keeps the gradient alive

Follow the gradient backwards along the highway only (ignore the paths through the gates).
One line of calculus:

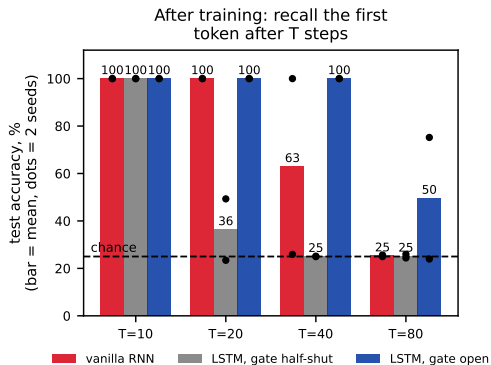
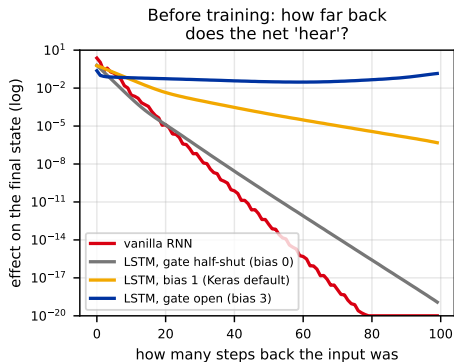
$$\mathbf{c}^{[t]} = \mathbf{f}^{[t]} \odot \mathbf{c}^{[t-1]} + \mathbf{i}^{[t]} \odot \tilde{\mathbf{c}}^{[t]} \implies \frac{\partial \mathbf{c}^{[t]}}{\partial \mathbf{c}^{[t-1]}} = \text{diag}(\mathbf{f}^{[t]})$$

Vanilla RNN (L20): each step multiplies by $\mathbf{D}^{[t-1]}\mathbf{V}^\top$ - the *same* matrix every step, whether or not the memory matters. Its size decides the fate of everything.

LSTM highway: each step multiplies by $\mathbf{f}^{[t]}$ - a gate the network *recomputes* from the input at every step, per memory slot. It can hold $f \approx 1$ while a memory matters (it survives 100 steps) and drop to $f \approx 0$ to wipe it on purpose.

History: the 1997 LSTM had *no* forget gate - its cell passed the gradient back with factor exactly 1 (a "constant error carousel"). The forget gate came in 2000 (Gers, Schmidhuber & Cummins), so the memory could also be cleared.

Does it work? Measured



Left, before any training: an input 40 steps back barely moves a vanilla RNN, nor an LSTM whose forget gates start half-shut; bias 1 still decays, only slower; open the gates (bias 3) and it still counts after 100 steps. **Right**, same budget for all: the open-gate LSTM solves $T \leq 40$ within 100 steps; the RNN is a coin flip at 40, at chance by 80. At 80 even the LSTM succeeds on one seed of two - addition makes long memory *learnable*, not free.

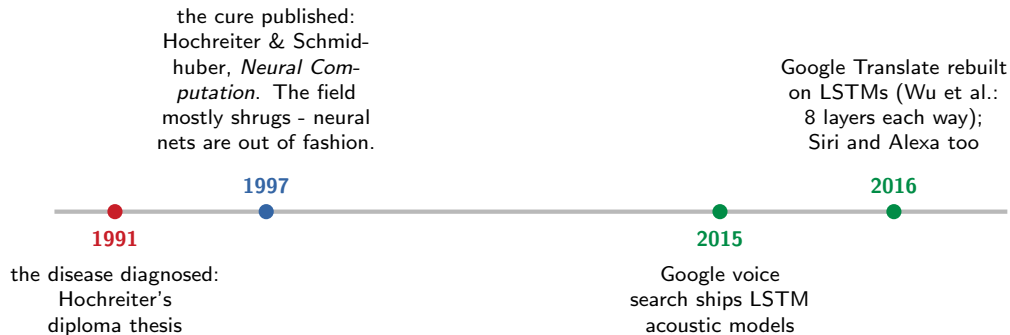
The price, and one gotcha

- ▶ **Price:** 4 weight sets instead of 1. Our $d_x=1, d_h=16$ vanilla cell: $16+256+16 = 288$ params. LSTM: $4 \times 288 = \mathbf{1,152}$.
- ▶ **Gated Recurrent Unit (GRU;** Cho et al., 2014): a streamlined 2-gate cousin (3 weight sets), similar results - the lighter default.
- ▶ **Gotcha:** the left panel's "gate starts open" is a one-line choice. Keras adds +1 to the forget-gate bias by default; **PyTorch does not:**

```
lstm = nn.LSTM(d_in, hidden)
# PyTorch stacks the gates: input, forget, cell, output
lstm.bias_ih_l0.data[hidden:2 * hidden].fill_(1.0) # forget gate
```

Same lesson as [45]'s initialization frames: where training *starts* can decide whether it gets anywhere at all.

Published 1997, everywhere by 2016



18 years from publication to running in everyone's pocket - and one of the most-cited AI papers ever (80,000+ citations). When a 2010s product said "powered by neural networks", the network was very often an LSTM.

Fine print, for next chapter: 2016's Google Translate already carried an *attention* link between its encoder and decoder - the idea beyond LSTM was creeping in even at LSTM's moment of triumph.

Before any of this works...

...text must become numbers. The kit attention will need, built here.

The tokenization trilemma

The review sentence, split three ways:

"The pomegranates arrived fresh and sweet." split three ways

characters (41 tokens, vocab $\approx$ 100)

T	h	e	_	p	o	m	e	g	r	a	n	a	t	e	s	_	a	r	r	i	v	e	d	_	f	r	e	s	h	_	a	n	d	_	s	w	e	e	t	.
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

words (7 tokens, vocab explodes; "pomegranatey" = OOV)

The	pomegranates	arrived	fresh	and	sweet	.
-----	--------------	---------	-------	-----	-------	---

subwords (real BPE tokenizer) (10 tokens, vocab $\approx$ 100k)

The	_p	ome	gran	ates	_arrived	_fresh	_and	_sweet	.
-----	----	-----	------	------	----------	--------	------	--------	---

Characters: tiny vocabulary, but sequences get long and a letter carries no meaning on its own.
Words: meaningful units, but the vocabulary explodes - and "pomegranatey" was never in it (out-of-vocabulary, OOV). **Subwords:** the compromise every modern model uses - frequent chunks become tokens, rare words split into pieces.

Subwords on a real tokenizer

The same sentence through an actual production tokenizer (c1100k_base, the GPT-3.5/4 family), token boundaries and IDs visible:

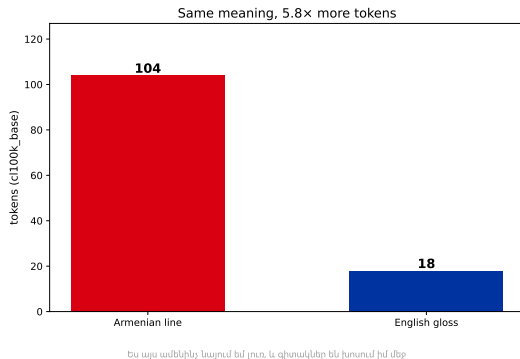
10 tokens for a 6-word sentence -- vocabulary built from frequent character pairs merged together (BPE, one breath; full mechanics: ch8)



The vocabulary is chosen by merging frequent character pairs (byte-pair encoding, BPE) - one breath here; the full merge mechanics belong to the LLM track that follows the attention chapter. Try it live: **tiktokenizer.vercel.app** - type Armenian and watch it fragment (next frame).

The Armenian tax

The chapter's Armenian line vs. its English gloss, through the *same* tokenizer - counts measured, not invented:

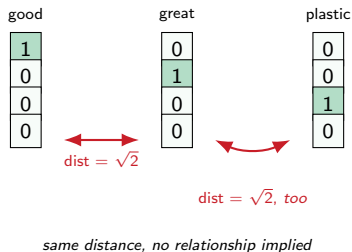


English-centric vocabularies fragment other scripts. Why it matters, twice over: longer sequences mean more steps, which means more fading (L20's disease again); and later, literally more money per API call - every token is billed.

One-hot and its poverty

Each token gets an ID; each ID, a one-hot vector. Callback: [03]'s one-hot encoding; L20's toy net used a 3-word version of exactly this.

In a 10,000-dimensional one-hot space, **every pair of words is equally far apart**. "good" is as far from "great" as it is from "plastic" - the geometry carries zero meaning.



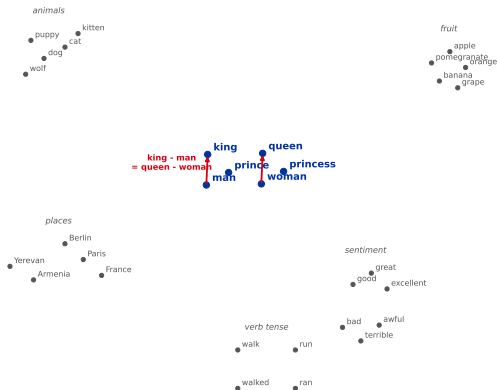
Embeddings

Cashing in L20's planted beat, **by name**: the anatomy frame showed one-hot $\times \mathbf{W}$ selecting a row of $\mathbf{W}$ - that row *is* the word's own vector. An **embedding table** is just that $\mathbf{W}$, grown to vocabulary size and learned.

Dense, learned vectors: similar words end up close: the famous king-man+woman $\approx$ queen arithmetic.

Trained once on huge corpora (word2vec, GloVe) and reused - a warm start, i.e. L18's transfer learning, for text. Limitation in one line: "bank" gets ONE vector regardless of context - the attention chapter scratches that itch. Explore: projector.tensorflow.org.

Illustrative schematic of a word-embedding space (2-D projection)
real embeddings: 100-1000 dimensions, learned from huge text corpora



One task to rule them all

Predict the next token. That is the whole objective.

The objective

$$P(y_1, \dots, y_T) = \prod_{i=1}^T P(y_i \mid y_1, \dots, y_{i-1})$$

Turns raw text into supervised data **for free**: every prefix is an input, every next token a label. No labeling budget, no annotators - just more text.

Said slowly, once: this exact objective, scaled up, is what GPT trains on.

"The pomegranates arrived fresh"



The → pomegranates

The pomegranates → arrived

The pomegranates arrived → fresh

every prefix → next-token pair, free supervision

The mechanics, click by click

Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 1 of 6

feeding **q**u a l i h p one character at a time

ILLUSTRATIVE -- not measured from a trained model

q u a l i h p

(distribution appears once the first character is fed)

no character fed yet -- the net has only its initial state

The mechanics, click by click

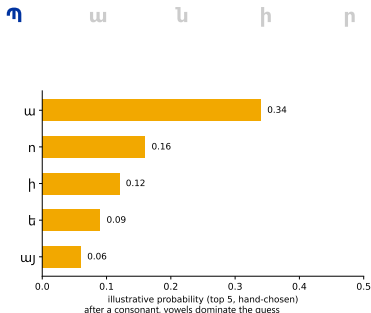
Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 2 of 6

feeding **q**ulk

h

ILLUSTRATIVE -- not measured from a trained model



The mechanics, click by click

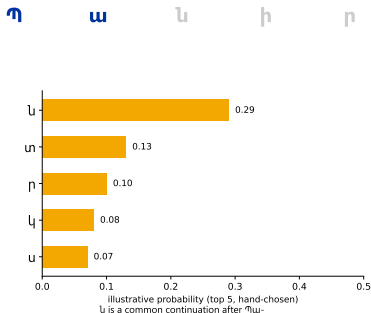
Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 3 of 6

feeding **q**ulh

one character at a time

ILLUSTRATIVE -- not measured from a trained model



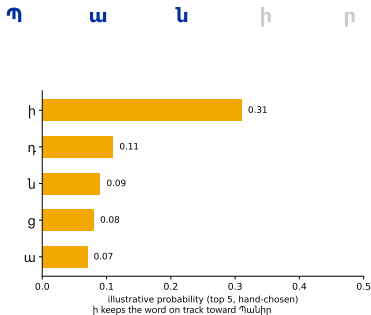
The mechanics, click by click

Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 4 of 6

feeding **q**ulh

ILLUSTRATIVE -- not measured from a trained model



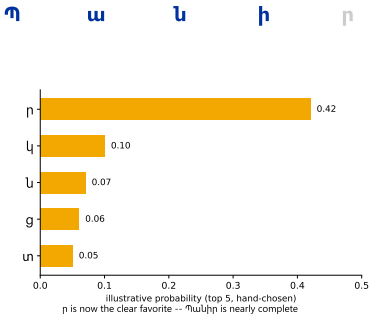
The mechanics, click by click

Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 5 of 6

feeding "c" one character at a time

ILLUSTRATIVE -- not measured from a trained model



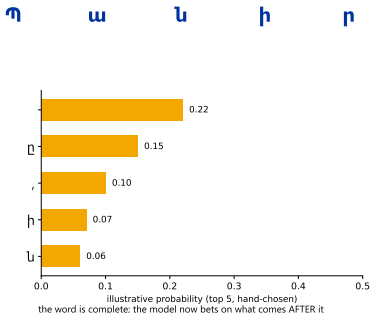
The mechanics, click by click

Feed a word's characters one at a time; at every step the net outputs a probability distribution over the *next* character (course cheese joke, five letters, shown below). **(Click to advance.)**

step 6 of 6

feeding Պաշկը one character at a time

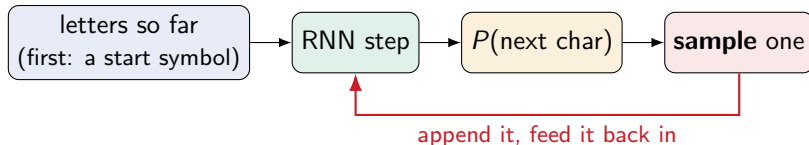
ILLUSTRATIVE -- not measured from a trained model



No repeated character here, so no "double letter" surprise to spring - one remark instead: the word's third letter also appears in several other words of the chapter's Armenian line. The model's bet after that letter depends on what came before it, not on the letter alone.

From predicting to writing: sample, feed back, repeat

The network only ever **predicts** one distribution over the next character. *Writing* is a loop around it:



Repeat until the net samples an end symbol. Why *sample* rather than always take the most likely character? The top choice every time gives the same, often repetitive text on every run; sampling gives variety. A **temperature** knob (divide the scores by T before the softmax) slides from safe to surprising - the knob from the ch11 name inventor.

This loop is exactly how ChatGPT writes: one token at a time, each one sampled and fed back in. Only the network inside the loop changes.

Demo: a char-LSTM learns to write Armenian

What a character-level LSTM trained on Armenian text typically produces, one checkpoint per click (illustrative progression, not a measured run): gibberish → letter statistics → word-like fragments → almost-literary → a seeded continuation. **(Click to advance.)**

checkpoint 1 of 5

ILLUSTRATIVE -- typical progression, not a measured run

a char-LSTM learns to write Armenian (typical progression)

step 0 (untrained)



թգխղրջ անխաուրծ խքացնեռ ընթղմա ցվատեղծ ոքրիմբ

pure noise -- random characters from the alphabet

Demo: a char-LSTM learns to write Armenian

What a character-level LSTM trained on Armenian text typically produces, one checkpoint per click (illustrative progression, not a measured run): gibberish → letter statistics → word-like fragments → almost-literary → a seeded continuation. **(Click to advance.)**

checkpoint 2 of 5

ILLUSTRATIVE -- typical progression, not a measured run

a char-LSTM learns to write Armenian (typical progression)

step ~100



անի որնա տեմա անու Լանիս որա Լամի անե

letter statistics emerge -- vowel-consonant rhythm, no real words yet

Demo: a char-LSTM learns to write Armenian

What a character-level LSTM trained on Armenian text typically produces, one checkpoint per click (illustrative progression, not a measured run): gibberish → letter statistics → word-like fragments → almost-literary → a seeded continuation. **(Click to advance.)**

checkpoint 3 of 5

ILLUSTRATIVE -- typical progression, not a measured run

a char-LSTM learns to write Armenian (typical progression)

step ~1,000



նա ամեն լավ խոսք չէր գտնում մեջքով վարդի աչքով

word-like fragments -- some real short words, invented ones in between

Demo: a char-LSTM learns to write Armenian

What a character-level LSTM trained on Armenian text typically produces, one checkpoint per click (illustrative progression, not a measured run): gibberish → letter statistics → word-like fragments → almost-literary → a seeded continuation. **(Click to advance.)**

checkpoint 4 of 5

ILLUSTRATIVE -- typical progression, not a measured run

a char-LSTM learns to write Armenian (typical progression)

step ~5,000



Ես տեսնում եմ այս ամենը լուռ ու մտածում իմ մասին

almost-literary -- real words, loose grammar, a plausible-sounding line

Demo: a char-LSTM learns to write Armenian

What a character-level LSTM trained on Armenian text typically produces, one checkpoint per click (illustrative progression, not a measured run): gibberish → letter statistics → word-like fragments → almost-literary → a seeded continuation. **(Click to advance.)**

checkpoint 5 of 5

ILLUSTRATIVE -- typical progression, not a measured run

a char-LSTM learns to write Armenian (typical progression)

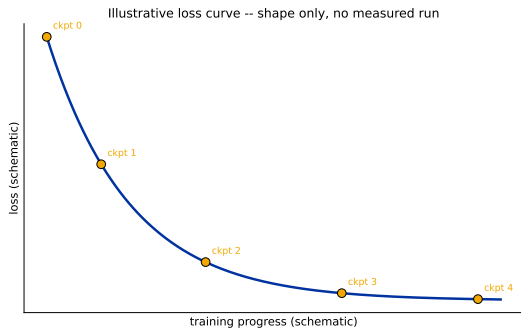
final (seeded)



Ես այս ամենիկնչ նայում եմ լուռ ու մտքումս խոսում եմ ինքս ինձ հետ

seeded with the chapter's opening words -- the model continues the line

The loss curve, and a link



Loss falls as the checkpoints improve (schematic curve) - the same next-token objective from two frames ago, watched from the outside.

This demo is the Armenian edition of a famous post: Andrej Karpathy, "**The Unreasonable Effectiveness of Recurrent Neural Networks**" (karpathy.github.io/2015/05/21/rnn-effectiveness/) - char-RNNs writing Shakespeare, Linux source, and Wikipedia markup, all from the same one objective.

The most visual proof in this course that next-token prediction alone extracts real structure from raw text.

Two networks, one vector between them

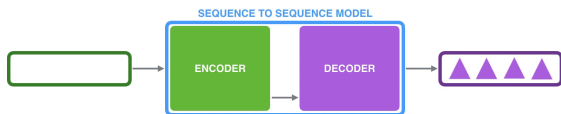
Translation needs a different-length output. Here is the classic fix.

Encoder-decoder

Translation needs different input/output lengths. Fix: an **encoder** LSTM reads the source and hands ONE **context vector** C to a **decoder** LSTM that writes the target.

The 2014 recipe that made neural machine translation real: Sutskever, Vinyals & Le (2014); Cho et al. (2014).

Running example: "I watch all of this in silence, and the connoisseurs speak inside me." goes in - and the decoder writes out the chapter's Armenian line, the sentence you just watched cost 104 tokens.



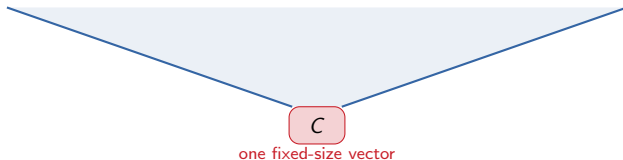
The bottleneck

Predict: the 42-word complaint review (L20's shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?

The bottleneck

Predict: the 42-word complaint review (L20's shoehorn-hacks frame - it lost its verdict to truncation already) squeezed into a single 300-dim context vector. What goes wrong?

42 words: "...expecting the same deep red arils and rich tart sweetness... arrived bruised dry and honestly disappointing"



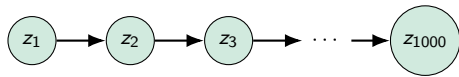
Lost: early words get overwritten as the state updates sequentially - "expecting", "childhood markets" fade first

Fixed capacity: 5 words or 500, the same-size C must hold them all

And the LSTM does NOT save us: the cell state is still fixed-size vector. The fix for *fading* memory did not fix *compressed* memory.

The second wall: one step at a time

Even a perfect LSTM has one more problem, and it is about **speed**, not memory: $\mathbf{z}^{[t]}$ needs $\mathbf{z}^{[t-1]}$, which needs $\mathbf{z}^{[t-2]}$... Step t cannot start until step $t - 1$ has finished.



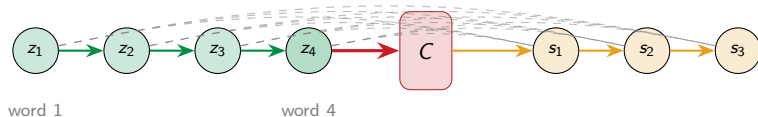
1,000 tokens = 1,000 steps
in a queue, one after another

- ▶ The GPUs from [42] want thousands of *independent* multiply-adds at once. A recurrence hands them one small step at a time - most of the chip waits.
- ▶ The authors of the paper that ended the RNN era put it plainly: "This inherently sequential nature precludes parallelization within training examples" (Vaswani et al., 2017).

Two walls, then: the **bottleneck** (one vector must hold everything) and the **queue** (one step at a time). The next chapter's idea removes both at once.

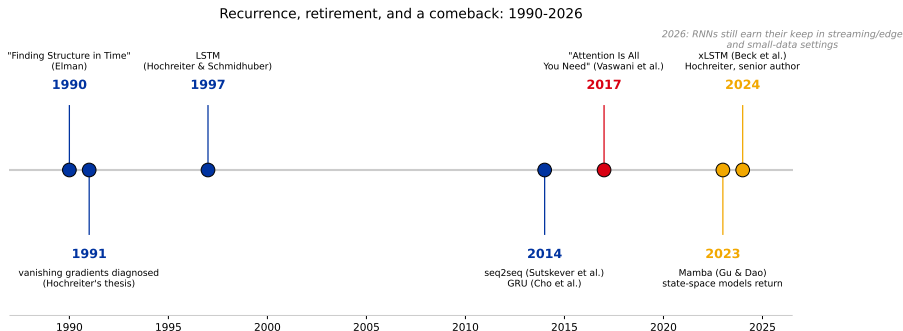
The cliffhanger

What if the decoder could look back at *all* the encoder states - and learn *where* to look?



That question has a name. It was answered between 2014 and 2017, and it ends this course's pre-history of sequence models. Next chapter: attention and transformers.

Recurrence, retirement, and a comeback



2017's transformer retired RNNs from mainstream NLP - but recurrence came back: state-space models (Mamba), **Hochreiter's own** xLSTM, and by 2025-26 frontier models - **Qwen3-Next** (Sept 2025) and **Qwen3.5** (Feb 2026) run three of every four layers as a fixed-size recurrent state (Gated DeltaNet) with a learned *decay* gate - today's forget gate, cheap on long inputs. Plain RNNs still earn their keep in streaming/edge and small-data work.

Recap

- ▶ Length varies, memory fades (L20) → LSTM fixes fading with an **additive** cell state - same medicine as ResNet, 18 years earlier - at $4\times$ the parameter cost.
- ▶ Text becomes numbers through **tokenization**: char / word / subword, and the Armenian line costs several times more tokens than its English gloss.
- ▶ One-hot is geometry-blind; **embeddings** are one-hot's learned, dense upgrade - the anatomy frame's "row of **W**," grown up.
- ▶ One objective - **predict the next token** - powers a char-LSTM writing Armenian and, at a different scale, GPT; **writing** is the loop sample → feed back → repeat.
- ▶ **Seq2seq** squeezes an entire source sentence into one vector - and even the LSTM's fix for fading memory cannot fix a memory that is *compressed*. And any recurrence runs **one step at a time**.

Next: the decoder wants to look back at everything. Letting it do that - attention - is where the attention chapter begins (L24); dropping the recurrence altogether also removes the one-step-at-a-time queue.